LAB 10 - Graph Algorithms: MST and Shortest Path

Data Structures and Algorithms Laboratory

Student Details

Name	Pratyush Deo Singh
Registration Number	25BCE5101
Semester	2nd
Subject	Data Structures and Algorithms
Slot	A2
Faculty Name	Dr. Malini A

Programs

Program 1: Prim's Algorithm - MST using Adjacency Matrix

File Name:

q1_prims.c

Program Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int vertices;
int adjMatrix[MAX][MAX];
int key[MAX];
int parent[MAX];
int inMST[MAX];

int minKey() {
    int min = INT_MAX, minIndex = -1;
    for (int v = 0; v < vertices; v++) {
        if (!inMST[v] && key[v] < min) {
            min = key[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void primMST() {
    for (int i = 0; i < vertices; i++) {
        key[i] = INT_MAX;
```

```
        inMST[i] = 0;
        parent[i] = -1;
    }

    key[0] = 0;

    for (int count = 0; count < vertices - 1; count++) {
        int u = minKey();
        inMST[u] = 1;

        for (int v = 0; v < vertices; v++) {
            if (adjMatrix[u][v] && !inMST[v] && adjMatrix[u][v] < key[v]) {
                key[v] = adjMatrix[u][v];
                parent[v] = u;
            }
        }
    }

    int totalCost = 0;
    printf("\nMST Edges (Prim's Algorithm):\n");
    printf("Edge\t\tWeight\n");
    for (int i = 1; i < vertices; i++) {
        printf("%d - %d\t\t%d\n", parent[i], i, adjMatrix[parent[i]][i]);
        totalCost += adjMatrix[parent[i]][i];
    }
    printf("Total Cost of MST: %d\n", totalCost);
}

int main() {
    int edges, u, v, w;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < vertices; i++)
        for (int j = 0; j < vertices; j++)
            adjMatrix[i][j] = 0;

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d %d", &u, &v, &w);
        adjMatrix[u][v] = w;
        adjMatrix[v][u] = w;
    }

    primMST();

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 7

Enter edges (u v weight):

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

Expected Output:

MST Edges (Prim's Algorithm):

Edge	Weight
0 - 1	2
1 - 2	3
1 - 4	5
0 - 3	6

Total Cost of MST: 16

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9

MST Edges (Prim's Algorithm):
Edge            Weight
0 - 1           2
1 - 2           3
0 - 3           6
1 - 4           5
Total Cost of MST: 16
```

Program 2: Kruskal's Algorithm - MST using Sorted Edges

File Name:

q2_kruskals.c

Program Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Edge {
    int src, dest, weight;
};

struct Edge edges[MAX * MAX];
struct Edge mst[MAX];
int parent[MAX];
int rank[MAX];
int edgeCount = 0;

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unionSets(int x, int y) {
    int px = find(x);
    int py = find(y);

    if (rank[px] < rank[py])
        parent[px] = py;
    else if (rank[px] > rank[py])
        parent[py] = px;
    else {
        parent[py] = px;
        rank[px]++;
    }
}

int compareEdges(const void* a, const void* b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
    int vertices, numEdges, u, v, w;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &numEdges);

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < numEdges; i++) {
        scanf("%d %d %d", &u, &v, &w);
```

```
        edges[i].src = u;
        edges[i].dest = v;
        edges[i].weight = w;
    }

    qsort(edges, numEdges, sizeof(struct Edge), compareEdges);

    for (int i = 0; i < vertices; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    int mstEdges = 0;
    int totalCost = 0;

    for (int i = 0; i < numEdges && mstEdges < vertices - 1; i++) {
        int pu = find(edges[i].src);
        int pv = find(edges[i].dest);

        if (pu != pv) {
            mst[mstEdges++] = edges[i];
            totalCost += edges[i].weight;
            unionSets(pu, pv);
        }
    }

    printf("\nMST Edges (Kruskal's Algorithm):\n");
    printf("Edge\t\tWeight\n");
    for (int i = 0; i < mstEdges; i++) {
        printf("%d - %d\t\t%d\n", mst[i].src, mst[i].dest, mst[i].weight);
    }
    printf("Total Cost of MST: %d\n", totalCost);

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 7

Enter edges (u v weight):

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

Expected Output:

MST Edges (Kruskal's Algorithm):

Edge	Weight
0 - 1	2
1 - 2	3
1 - 4	5
0 - 3	6

Total Cost of MST: 16

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9

MST Edges (Kruskal's Algorithm):
Edge            Weight
0 - 1           2
1 - 2           3
1 - 4           5
0 - 3           6
Total Cost of MST: 16
```

Program 3: Dijkstra's Algorithm - Shortest Path from Source

File Name:

q3_dijkstra.c

Program Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int vertices;
int adjMatrix[MAX][MAX];
int dist[MAX];
int visited[MAX];
int prev[MAX];

int minDist() {
    int min = INT_MAX, minIndex = -1;
    for (int v = 0; v < vertices; v++) {
        if (!visited[v] && dist[v] < min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void printPath(int dest) {
    if (prev[dest] == -1) {
        printf("%d", dest);
        return;
    }
    printPath(prev[dest]);
    printf(" -> %d", dest);
}

void dijkstra(int src) {
    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
        prev[i] = -1;
    }

    dist[src] = 0;

    for (int count = 0; count < vertices - 1; count++) {
        int u = minDist();
        if (u == -1)
            break;
        visited[u] = 1;

        for (int v = 0; v < vertices; v++) {
            if (!visited[v] && adjMatrix[u][v] && dist[u] != INT_MAX &&
                dist[u] + adjMatrix[u][v] < dist[v]) {
                dist[v] = dist[u] + adjMatrix[u][v];
                prev[v] = u;
            }
```

```
        }
    }
}

int main() {
    int edges, u, v, w, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < vertices; i++)
        for (int j = 0; j < vertices; j++)
            adjMatrix[i][j] = 0;

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d %d", &u, &v, &w);
        adjMatrix[u][v] = w;
        adjMatrix[v][u] = w;
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(src);

    printf("\nShortest distances from vertex %d:\n", src);
    printf("Vertex\t\tDistance\n");
    for (int i = 0; i < vertices; i++) {
        if (dist[i] == INT_MAX)
            printf("%d\t\tINFINITY\n", i);
        else
            printf("%d\t\t%d\n", i, dist[i]);
    }

    printf("\nEnter destination vertex: ");
    scanf("%d", &dest);

    if (dist[dest] == INT_MAX) {
        printf("No path from %d to %d.\n", src, dest);
    } else {
        printf("Shortest Path from %d to %d: ", src, dest);
        printPath(dest);
        printf("\nTotal Distance: %d\n", dist[dest]);
    }

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 7

Enter edges (u v weight):

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 7

3 4 9

Enter source vertex: 0

Enter destination vertex: 4

Expected Output:

Shortest distances from vertex 0:

Vertex	Distance
0	0
1	2
2	5
3	6
4	7

Shortest Path from 0 to 4: 0 -> 1 -> 4

Total Distance: 7

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
Enter source vertex: 0

Shortest distances from vertex 0:
Vertex          Distance
0               0
1               2
2               5
3               6
4               7

Enter destination vertex: 4
Shortest Path from 0 to 4: 0 -> 1 -> 4
Total Distance: 7
```